

Name- Bhumika Shrotriya

Course Name – Data Analytics With GenAI

Subject – Data Processing & ETL

Topic – Data Quality & Validation

Question 1: Define Data Quality in the context of ETL pipelines. Why is it more than just data cleaning?

Answer 1:

Data Quality in ETL refers to the degree to which data is accurate, complete, consistent, timely, valid, and fit for its intended purpose throughout the Extract, Transform, and Load process.

It is **more than just data cleaning** because:

- **Data Cleaning** is reactive — it fixes known issues like nulls, typos, or formatting errors.
- **Data Quality** is proactive and holistic — it encompasses:
 - **Completeness:** Are all required fields present?
 - **Consistency:** Is data consistent across systems (e.g., same customer name in CRM and billing)?
 - **Validity:** Does data conform to business rules (e.g., Quantity must be > 0)?
 - **Uniqueness:** Are there duplicate records?
 - **Timeliness:** Is data available when needed?
 - **Referential Integrity:** Do foreign keys point to valid records?

For example, in our dataset, Txn_ID 206 has Customer_Name = N/A and Txn_Amount = Null. Cleaning might fill N/A with "Unknown", but a data quality framework would flag why this customer (C105) doesn't exist in the master table — a deeper integrity issue.

Question 2: Explain why poor data quality leads to misleading dashboards and incorrect decisions.

Answer 2:

Poor data quality propagates errors through the entire analytical pipeline:

Issue	Example from Dataset	Impact on Dashboard
Duplicate records	Txn 201, 203, 208 are identical	Revenue counted 3× instead of 1×
Null values	Txn 206: Txn_Amount = Null	Total revenue understated
Missing dates	Txn 207: Txn_Date = Null	Excluded from time-series trends
Invalid customers	C105, C106 not in master	Orphan records inflate customer count
Null quantity	Txn 205: Quantity = Null	Wrong average order size

Real-world consequence: If a sales dashboard shows ₹19,800 total revenue (counting all rows including duplicates), management may reward a region incorrectly. The true deduplicated revenue is significantly lower. Decisions built on such data — budgeting, staffing, inventory — will all be wrong.

Question 3: What is duplicate data? Explain three causes in ETL pipelines.

Answer 3:

Duplicate data refers to records that represent the same real-world entity or event more than once in a dataset, either identically or with slight variations.

Three causes in ETL pipelines:

- 1. Re-running ETL Jobs Without Deduplication Logic** When an ETL pipeline is re-executed (e.g., due to failure or scheduled re-run), it may re-extract and reload the same source data without checking if records already exist. In our dataset, Txn_IDs 201, 203, and 208 appear to be the result of such re-processing — same Customer_ID, Product_ID, Quantity, Amount, and Date.
- 2. Multiple Source Systems Feeding the Same Target** When data is ingested from multiple sources (e.g., POS system + mobile app + CRM), the same transaction may be recorded in more than one system and loaded without cross-source deduplication.
- 3. Lack of Unique Key Enforcement at Load** If the target table does not enforce a PRIMARY KEY or UNIQUE constraint, the same record can be inserted multiple times. Without a surrogate key check or upsert logic, duplicates silently accumulate.

Question 4: Differentiate between exact, partial, and fuzzy duplicates

Answer 4:

Type	Definition	Example
Exact Duplicate	All field values are identical across two or more records	Txn 201, 203, 208 — same Customer_ID, Product_ID, Quantity, Amount, Date, City
Partial Duplicate	Same business entity but some fields differ (e.g., different Txn_ID or timestamp)	Same customer buying same product on same day but with a slightly different Txn_Amount due to rounding
Fuzzy Duplicate	Records that represent the same entity but have variations due to typos, abbreviations, or formatting differences	"Rahul Mehta" vs "R. Mehta" vs "Rahul Mehta " (trailing space) — same person, different string representation

Detection approaches:

- Exact: Simple GROUP BY + HAVING COUNT(*) > 1
- Partial: Composite key matching with tolerance on select fields

- Fuzzy: Algorithms like Levenshtein distance, Soundex, or ML-based entity resolution

Question 5: Why should data validation be performed during transformation rather than after loading?

Answer 5:

Performing validation **during transformation** (not after loading) is a best practice for these reasons:

- 1. Cost of Fixing Errors Increases Post-Load** Once bad data is loaded into a data warehouse or reporting layer, correcting it requires DELETE/UPDATE operations on production tables, potential downstream cache invalidation, and re-running dependent jobs.
- 2. Prevents Pollution of the Target System** The target (e.g., data warehouse, BI layer) should only contain clean, trusted data. Loading invalid records — even temporarily — can corrupt aggregations, materialized views, and pre-built reports.
- 3. Early Failure = Faster Debugging** When validation fails during transformation, the pipeline halts with a clear error message at the exact record and field. Post-load validation requires tracing back through loaded data to find the source.
- 4. Supports Data Lineage and Auditability** Transformation-stage validation can log rejected records to an error/quarantine table with metadata (source, timestamp, rejection reason), enabling full traceability.

Example: In our dataset, Txn 207 has Txn_Date = Null. Catching this during transformation allows routing it to a rejected_records table. If loaded, any date-based partitioning or time-series query would either fail or silently exclude the record.

Question 6: Explain how business rules help in validating data accuracy. Give an example.

Answer 6:

Business rules are domain-specific constraints that define what "valid" data looks like from a business perspective — beyond mere technical correctness.

They act as a semantic layer of validation:

Rule Type	Example
Range/Threshold	Txn_Amount > 0 and Txn_Amount < 1,000,000
Referential	Customer_ID must exist in Customers_Master
Conditional	If Quantity IS NULL, the transaction is incomplete and must be rejected
Format	Txn_Date must follow YYYY-MM-DD and cannot be a future date

Rule Type	Example
Mandatory fields	Customer_Name cannot be NULL or 'N/A'

Concrete Example from the dataset:

Rule: A transaction must have a valid Customer_ID that exists in Customers_Master, a non-null Txn_Amount > 0, a non-null Quantity > 0, and a valid Txn_Date.

Applying this rule flags:

- Txn 205 → Quantity = NULL
- Txn 206 → Txn_Amount = NULL, Customer_ID = C105 not in master
- Txn 207 → Txn_Date = NULL

Without business rules, these records would pass basic technical checks (they are valid rows in a table) but would produce incorrect analytics.

Question 7: SQL Query to Find Duplicate Business Keys

Answer 7:

```

SELECT
    Customer_ID,
    Product_ID,
    Txn_Date,
    Txn_Amount,
    COUNT(*) AS duplicate_count
FROM Sales_Transactions
GROUP BY
    Customer_ID,
    Product_ID,
    Txn_Date,
    Txn_Amount
HAVING
    COUNT(*) > 1
ORDER BY
    duplicate_count DESC;

```

Expected Output:

Customer_ID	Product_ID	Txn_Date	Txn_Amount	duplicate_count
C101	P11	2025-12-01	4000	3

Question 8: Detecting Referential Integrity Violations**Answer 8:**

SELECT DISTINCT

st.Txn_ID,

st.Customer_ID,

st.Customer_Name

FROM Sales_Transactions st

LEFT JOIN

Customers_Master cm

ON st.Customer_ID = cm.CustomerID

WHERE

cm.CustomerID IS NULL;

Expected Output:

Txn_ID	Customer_ID	Customer_Name
206	C105	N/A
207	C106	Amit Verma